When AI Debugs AI: Repair or Bypass? Separating Two Outcomes of Weight-Space Programming

Chandrahas Aroori
chandrahas.aroori@gmail.com

November 2026 (draft v0.1)

Abstract

"Weight-space programming" is the claim that a frontier language model, given a small model's weights and a budget of forward passes, can debug the small model: locate the cause of a behaviour and remove it by writing a direct weight edit, with no training. The only published success is a loop fix in Qwen3.5-0.8B on math, and public replications agree that its gain comes from answers that now finish, not from new ability. We show that "the loop is gone" covers two different outcomes. In a *repair* the model still reasons and its reasoning now ends. In a *bypass* the model closes its thinking block before any reasoning, which the chat template already offers as a flag. One measurement separates them: the amount of text before the `</think>` tag. We test the method on text-to-SQL, where the same model with thinking enabled loops on 91% of questions, with Claude Fable 5.1 as the editor in two independent sessions of 25 forward passes and 300 code runs each. Under greedy decoding, both sessions removed the loop on 150 held-out BIRD Mini-Dev questions (loop rate 90.7% to 26.0% and 24.0%; accuracy 0.7% to 6.0% and 5.3%) by edits of 4,083 and 2,048 weight entries on different neurons. Random-location edits of the same size do nothing. Both edits are bypasses: 150 of 150 answers close thinking with zero reasoning, and neither beats the template flag (7.3%). Gradient descent restricted to each footprint, trained toward the bypass, reaches it. The first session's two attempts at a repair, each built from a measured mechanistic hypothesis, failed. The published math edit is a repair by the same measurement. With the report's own sampling settings it removes most of the SQL loop as well (cut answers 75.3% to 8.0%; 139 of 150 answers close thinking after a median of 16,146 characters of reasoning), an effect that greedy decoding hides. The bypass edits do nothing on MATH-500. So an edit that ends the SQL loop without removing the reasoning exists and carries over from math, while the bypass does not carry over to math; the frontier editor, working under greedy decoding with a success criterion that a bypass satisfies, did not find the repair. We release the edits, all turn logs, the harness and the evaluation. [TODO: GPT-6 Astra runs and the ablation map]

1 Introduction

Takehi et al. [1] report that GPT-6 Astra, given a Python kernel over the weights of Qwen3.5-0.8B and a budget of forward passes, raised the model's MATH-500 score from 7.9% to 48.1% by subtracting a "loop direction" from 130,188 entries in two MLP down-projection matrices. They call the method weight-space programming. The result is striking because no gradient was involved: the editor read activations, formed a hypothesis about why the model repeated itself, and wrote the fix by hand. The authors' own follow-up on Gemma 3 270M, a model that does not loop, gained nothing, and they conclude that editing "could remove an obstacle that hid an ability, but it could not add an ability the model lacked."

This paper asks a narrower question than "does it work": *what does the method actually produce when it works?* Two readings of the math result are open. In the first, the editor understood a mechanism and repaired it. In the second, the editor found some small set of weights whose change happens to stop the loop, and the mechanistic story is decoration. The two readings predict different things for a new domain: a repair found by understanding should look like the explanation that produced it; a lever found by search need not. We use three words throughout. A *repair* changes what sustains a behaviour. A *bypass* stops the model entering the state in which the behaviour occurs. A *lever* is any small weight change that alters the behaviour, whatever the explanation. We call Takehi et al. "the report". An answer is *cut* if it reaches the token limit and *loops* if some 40-character window of it occurs five or more times. What sustains a behaviour cannot be read from outputs, so in the measurements we use an operational test: an answer shows a repair if the thinking block closes after non-empty text, and a bypass if it closes with none. Section 5 checks that the non-empty text is mostly not repetitive, but the test does not establish that the sustaining mechanism was changed, and our uses of "repair" for an edit carry that limit. We argue that in this instance the method produced a bypass. Public replications re-evaluate the released math edit, and one blind attempt reruns the method on math [6]; none runs it on a new task or separates the editor's explanation from the search that found the edit.

We set up an independent instance of the method with a different editor, a different task and a held-out benchmark the editor never saw, and record every hypothesis the editor stated, every edit it made, and whether the edit did what the hypothesis predicted. Our contributions:

1. A re-implementation of the harness (sandboxed kernel, forward-pass service with generation and teacher-forced modes, handoff protocol) that runs on one laptop.

2. A new arena, text-to-SQL, where Qwen3.5-0.8B with thinking enabled loops on 91% of questions and the authors' released math edit barely transfers under greedy decoding (cut rate 98.7% to 87.3%).

3. A full session trace in which a frontier editor stated three mechanistic hypotheses, tested three edits, and succeeded only with the third.

4. Controls that separate the claim into parts (Section 4; the harness and task are in Section 2, the three attempts in Section 3, the reading of the trace in Section 6): a template-flag control (what skipping thinking is worth), random-location edits of the same footprint (whether the neuron choice mattered), and a same-footprint gradient baseline (whether an optimiser finds the same solution). Gradient descent on the editor's 4,083 entries reproduces the bypass by a different direction. [TODO: plus an ablation map]

5. A measurement that separates repair from bypass, the amount of text before `</think>`, applied to every edit under greedy decoding and under the report's sampling protocol, on SQL and on MATH-500 (Section 5).

2 Setup

Target model. Qwen3.5-0.8B [2]: 24 layers, width 1,024, MLP width 3,584, vocabulary 248,320, tied embeddings. Six layers (3, 7, 11, 15, 19, 23) use full attention; the others use gated linear attention. Weights are bfloat16 safetensors; the editor reads and writes them as float32 NumPy arrays and the harness casts edits back to bfloat16.

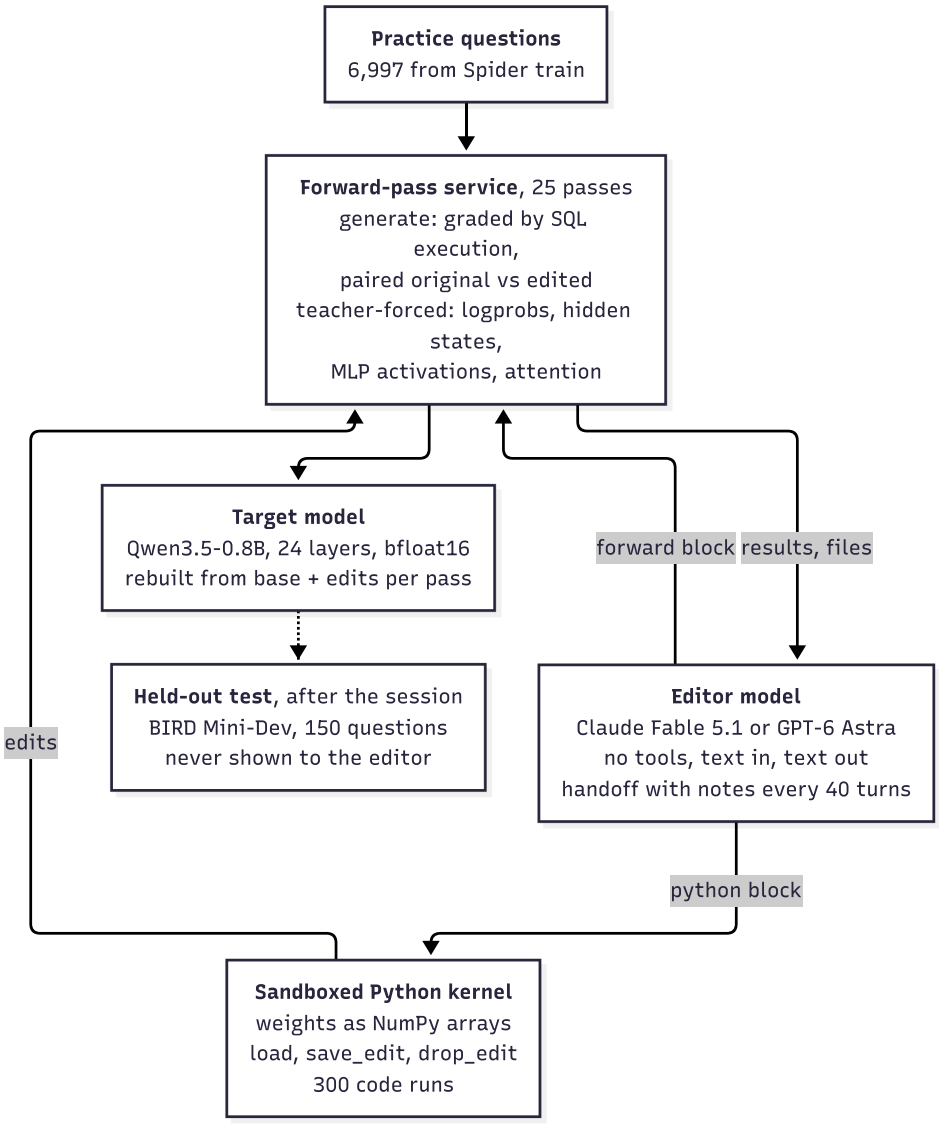


Figure 1: The harness. The editor model has no tools of its own; it writes Python blocks that run in a sandboxed kernel over the weights, and forward blocks that run the target model. Generation passes are graded by executing the produced SQL against the database; teacher-forced passes return log-probabilities, hidden states, MLP activations and attention. The held-out benchmark is never shown to the editor.

Task and obstacle. Text-to-SQL: the user message gives a SQLite schema and a question; the answer is graded by executing the produced query and comparing result sets with the gold query. With thinking enabled through the chat template, Qwen3.5-0.8B writes "Thinking Process: 1. Analyze the Request ... 2. Analyze the Schema ..." and then repeats schema bullets until the 8,192-token limit. On 150 BIRD Mini-Dev questions [7] (every third question, indices 0 to 447, covering all 11 databases), greedy decoding: 0.7% correct, 98.7% cut at the limit, 90.7% with a visible repetition (a 40-character window that occurs five or more times). This is the same failure class the report fixes on math, in a domain where, under greedy decoding, the report's edit mostly does not transfer (Table 1; Section 5 shows that this depends on the decoding).

Practice and test. The editor sees practice questions only: 6,997 from Spider train [8], with gold SQL and databases. The held-out test is BIRD Mini-Dev (500 questions over 11 databases); thinking-mode evaluations use 150 evenly spaced questions because each looping answer costs 8,192 tokens.

Budgets and rules. These follow the report: 25 forward passes, 300 code runs, a handoff every 40 turns in which the conversation restarts with the task, the editor's notes file and its current edit list, and a final answer when the passes are spent. A generation pass answers up to 128 practice questions (64 per model in a paired "both" request) with greedy decoding and 8,192 new tokens; a teacher-forced pass reads up to 1,024 texts. Gradient descent, running the model inside the kernel, and edits to the final norm, the embeddings, or whole-tensor rescaling are refused. The kernel is sandboxed (no network; writes only inside the run directory).

Editor. Claude Fable 5.1, called headless with tools, user settings and memory disabled, reasoning effort at its highest setting, one conversation per handoff. The report used GPT-6 Astra; we keep that path for the editor-dependence runs. [TODO: Astra runs]

Evaluation after the session. The submitted edit replaces the original tensors; the model is run with mlx-lm, greedy, 8,192 new tokens, thinking enabled, on the same 150 BIRD questions as the baseline. We report execution accuracy, the share of answers cut at the token limit, the loop share, and paired wrong-to-right and right-to-wrong counts against the base model on the same questions.

3 The session

The session ran 9.8 hours on one laptop (328 turns, 299 code runs, 25 passes, 9 handoffs; 493k output tokens from the editor). Figure 2 summarises it; the editor's notes file and all turns are released.

Attempt 1: induction heads. After one diagnostic generation pass (1.6% correct, 99% cut, 95% looping), the editor teacher-forced two loop transcripts with attention saved at the six full-attention layers. In the loop region it found heads that put attention mass one period back, the signature of induction heads [9]: layer 15 head 7 at 0.26 mass, then layer 11 head 1, layer 23 head 5 and layer 19 head 5. It also measured that the loop is "soft": at each bullet start the copied token leads the best alternative by a median 1.6 nats, and greedy decoding always takes the copy. Its edit zeroed the output-projection columns of the four heads. Paired generation on 40 fresh problems:

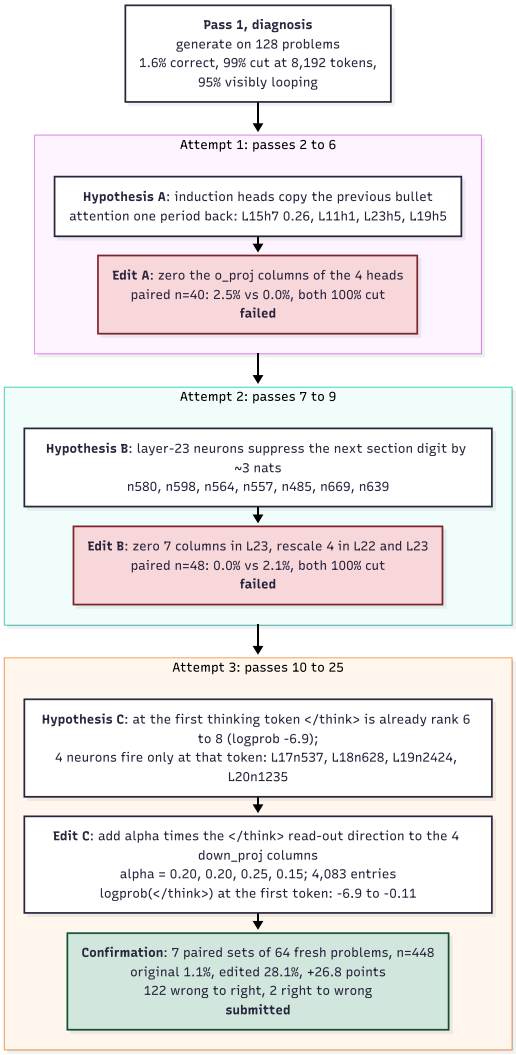


Figure 2: The 25-pass session: three hypotheses, three edits, one success. Pass numbers refer to the forward-pass budget; code runs and handoffs are omitted. Numbers inside the boxes are the editor's own measurements on practice questions.

original 2.5%, edited 0.0%, both 100% cut. The editor dropped the edit. Layer 15 head 7 is, as it happens, the head the report's p25c run zeroed for math; here it did nothing.

Attempt 2: digit-suppressing neurons. The editor then teacher-forced six of the model's own loop transcripts with hidden states and MLP activations in layers 16 to 23, and asked a sharper question: at each blank line the model chooses between "indent and add another bullet" and "write the next section number". It found seven neurons in the last layer's MLP that fire when a digit is about to be predicted and push that exact digit down by about 3 nats (n580 for '3', n598 for '5', n564 for '4', n557 for '1', n485 for '2', n669 for '6', n639 for '7'), plus three neurons that move the newline decision. Its edit zeroed the seven columns and rescaled four others in layers 22 and 23. Paired generation on 48 fresh problems: original 0.0%, edited 2.1%, both 100% cut. Dropped. The editor's note at this point reads: "The looping is distributed and not fixable at one site."

Attempt 3: the start-of-thinking switch. The editor changed the question from "why does the loop continue" to "why does the model enter this mode at all". At the first thinking position, right after the `<think>` tag and newline, `</think>` is already a plausible continuation (log-probability -6.9, rank 6 to 8), and direct logit attribution shows the MLPs of layers 16 to 23 pushing against it diffusely, by about 9 nats in total. It then searched for neurons selective to that position and found four, one each in layers 17 to 20 (n537, n628, n2424, n1235), with activations 1.2 to 2.7 there and a 99.9th-percentile magnitude elsewhere of 0.3 to 0.7. The edit adds to each neuron's down-projection column a multiple α of the unit vector $u = \mathrm{unit}(\gamma \odot E[\texttt{</think>}])$, where γ is the final norm gain and E the tied unembedding, with $\alpha = 0.20, 0.20, 0.25, 0.15$ (Figure 3). 4,083 entries change; the largest change is 0.039.

Teacher-forced, the log-probability of `</think>` at the first position rose from -6.9 to between -0.11 and -0.24, including on a 4,196-token prompt and on BIRD-style prompts with a hint line. The editor then spent its remaining passes on confirmation: seven paired sets of 64 fresh practice problems, 448 in total, original about 1% and edited about 28%, a gain of 26.8 points with every set between 18.8 and 31.3 points and 3 to 5 standard errors above zero; 122 problems went wrong to right and 2 right to wrong; cut answers fell from about 100% to 3 to 16%. A variant that wrote the direction of the token 'Okay' instead, to start free-form thinking rather than close it, scored 3.1% with 98% cut, and was dropped.

4 Held-out results and controls

The edit transfers. On BIRD, which the editor never saw and whose prompts are longer and carry a hint line, edit C removes the thinking loop: cut answers fall from 98.7% to 27.3% and loops from 90.7% to 26.0%. Accuracy rises from 0.7% to 6.0%, with 9 questions wrong to right and 1 right to wrong. The remaining cuts are loops inside the SQL itself (errors such as "too many FROM clause terms"), which the base model also produces with thinking off.

The neuron choice mattered. Five edits that apply the same four change vectors to randomly chosen neurons in the same four layers leave the model where it was: 0 to 2% correct, 100% cut, 90 to 98% looping. The editor picked four neurons out of 14,336 candidates in those layers and the pick was not interchangeable.

The edit is worth about one template flag. The base model with thinking disabled through the chat template, same 150 questions and token limit, scores 7.3% with 12.0% cut and 10.7%

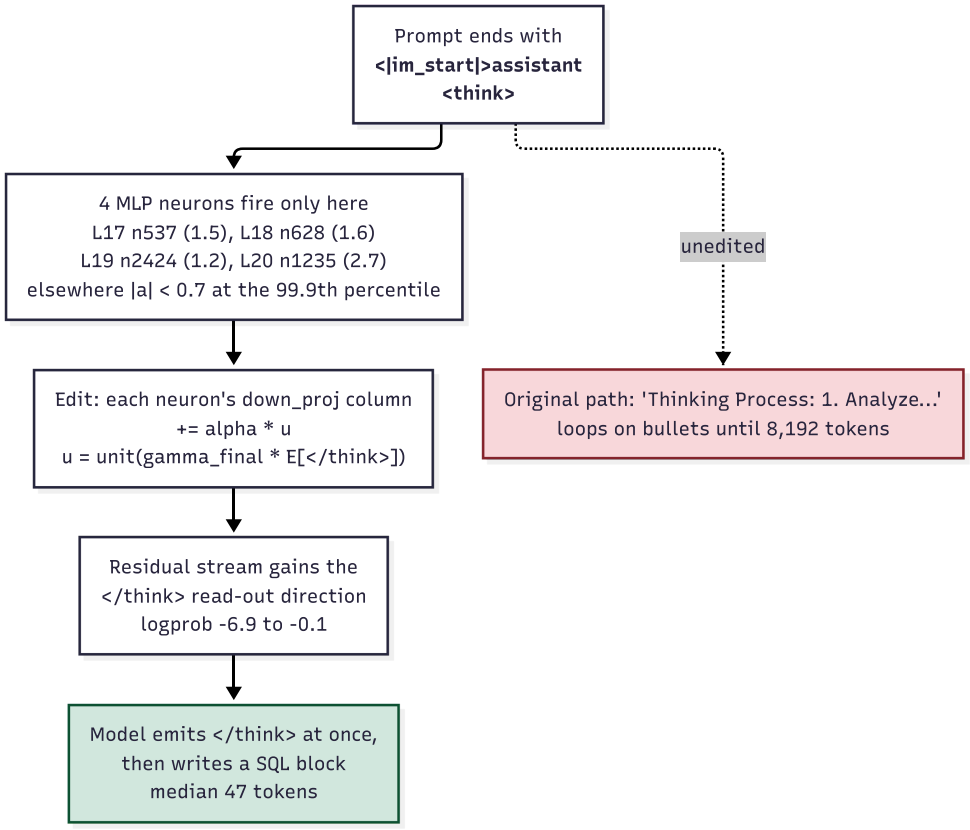


Figure 3: The successful edit. Four MLP neurons that fire only at the first thinking token are made to write the `</think>` read-out direction; the model closes its thinking at once and writes SQL.

Condition	Entries	n	Acc. %	Cut %	Loop %	W→R / R→W
Base, thinking on	0	150	0.7	98.7	90.7	
Base, thinking off via template, 8,192 tokens	0	150	7.3	12.0	10.7	
Base, thinking off, 1,024 tokens, all questions	0	500	7.0	14.0	9.0	
Authors' math edit [1], thinking on	130,188	150	3.3	87.3	82.0	5 / 1
Fable edit C (this work), thinking on	4,083	150	**6.0**	27.3	26.0	9 / 1
Random location, same footprint, 5 draws	4,083	5×50	0.0 to 2.0	100	90 to 98	
Gradient descent on the same 4,083 entries	4,083	150	4.7	30.7	29.3	7 / 1
Fable run 2 edit, thinking on	2,048	150	5.3	25.3	24.0	8 / 1
Random location, run 2 footprint, 5 draws	2,048	5×50	0.0 to 4.0	94 to 100	86 to 96	
Gradient descent on the run 2 footprint	2,048	150	7.3	29.3	29.3	11 / 1
Fable edit A (induction heads), practice	[TODO:]	40	0.0	100	100	
Fable edit B (digit neurons), practice	[TODO:]	48	2.1	100	100	

Table 1: BIRD Mini-Dev, greedy, 8,192 new tokens unless stated, the same 150 evenly spaced questions in every 150-question row. "Cut" is the share of answers that hit the token limit; "Loop" the share with a visible repetition. The last column is the paired count of questions that changed outcome against the base model on the same questions. Edits A and B were only evaluated on practice questions inside the session. [TODO: two Astra runs as further rows]

Condition (thinking on, 150 questions)	Closes thinking	With zero reasoning	Median chars before </think>
Base	2	0	3,092
Authors' math edit [1]	22	0	5,768
Fable run 1 edit	150	150	0
Fable run 2 edit	150	150	0
Gradient descent, run 1 footprint	150	150	0
Gradient descent, run 2 footprint	150	150	0

Table 2: How each edit makes the loop go away, greedy decoding. "Closes thinking" counts answers that contain </think>; the median is over those answers.

looping. Accuracy is indistinguishable from the edit (6.0% against 7.3%, $n = 150$), but the edit leaves about twice as many cut answers; we have not explained that gap. The chat template with thinking off inserts an empty <think></think> block, and the edit makes the model emit the closing tag itself, so the two conditions differ in whether the tag comes from the template or from the weights. The edited weights also stay in place while the SQL is written, which may account for the extra cut answers.

Gradient descent can realise the same bypass on the same entries. We trained only the same 4,083 entries with Adam (250 steps, batch 2, learning rate 2×10^{-4}, rows capped at 1,024 tokens) on practice questions, with the target </think> followed by the gold SQL. The result behaves like edit C: every one of the 150 BIRD answers opens with </think>, accuracy is 4.7%, cut 30.7%, loop 29.3%, paired 7 wrong to right and 1 right to wrong. The change vectors are about twice as long as the editor's (norms 0.38 to 0.41 against 0.15 to 0.25) and nearly orthogonal to its direction u (cosine 0.01 to 0.12). The training target prescribes the bypass, so this baseline shows that the footprint is sufficient for an optimiser to realise it, not that an optimiser would choose a bypass unprompted. Given the footprint and the target, an optimiser reaches the same behaviour by a different route; what the editor contributed that the optimiser did not is the choice of the four neurons.

A second session converges on the bypass, not on the neurons. A second Fable session, with a fresh conversation and no shared notes, also concluded that the loop could not be removed at one site and also went for the entry into thinking. It replaced the output column of one neuron, layer 18 neuron 2800, which fires only at the first thinking position, with a direction that drives </think>, and wrote a constant steering direction (the residual-stream difference induced by an in-context instruction to keep the query simple) into layer 16 neuron 3, which fires on every token. On 192 fresh practice problems: 1.6% to 28.1%, 51 wrong to right, 0 right to wrong. On BIRD: 5.3% correct, 25.3% cut, 24.0% looping. None of its neurons is one of the first session's four. Random-location draws do nothing, and gradient descent on its 2,048 entries again realises the bypass (7.3%, every answer opening with </think>).

5 Separating repair from bypass

Table 2 is the measurement the title refers to. An edit that repairs the loop leaves reasoning in place and lets it end; an edit that bypasses it removes the reasoning. The count of characters before </think> tells them apart at no extra cost, from outputs every evaluation already has. The

authors' math edit, which was not written for SQL and under greedy decoding helps little here, is nonetheless of the first kind: all 22 answers that close thinking do so after non-empty text. All four edits found on SQL, two by a frontier editor and two by gradient descent, are of the second kind. A second signature is expected: a pure bypass should match the template flag it imitates, up to side effects of the changed weights on later tokens, while a repair can exceed it. Independent evaluations of the math edit report exactly that on MATH-500, 50.2% edited with thinking on against 45.8% for the base model with thinking off [4], and the same evaluations and others [3, 5, 6] establish that the gain over the thinking-on baseline comes from answers that finish. Those analyses ask whether the edit adds ability; none asks which of the two outcomes it is.

Sampling, and transfer to math. All results above use greedy decoding. The report samples (temperature 0.6, top-p 0.95, top-k 20, presence penalty 1.5), so we reran the 150 held-out questions with those settings, and ran every edit on 100 MATH-500 problems (Table 3). Two things change. First, with its own sampling settings the authors' math edit removes most of the SQL loop: cut answers fall from 75.3% to 8.0%, and 139 of 150 answers close thinking, none at once, after a median of 16,146 characters of reasoning. Greedy decoding hid this effect. The text before `</think>` is mostly not a loop that happened to end: 9 of those 139 blocks contain a 40-character window repeated five or more times, and all 16 correct answers come from answers that closed thinking. Second, we see no effect of the Fable edits on math: no answer closes thinking at once, and about 90% of answers are cut, as for the base model (loop shares 73% base, 79% and 88% edited). On this evidence the bypass is specific to the SQL prompt, and the math edit carries over from math to SQL. The accuracy differences on SQL are not significant: 16, 10 and 8 correct of 150 for the math edit, thinking off and the base model, and 3 and 4 for the two Fable edits (on paired questions the math edit is right where the base model is wrong on 13 and the reverse on 5, exact McNemar $p \approx 0.10$). These runs use one sampling seed, and the random-location and gradient controls were not repeated under sampling. The math edit's 21% is below the published 48%; our limit is 16,384 tokens, half the report's, and 72 of 100 answers reach it, which we take to be the cause but have not tested at a matched limit. The thinking-off rows use the model card's sampling settings (temperature 0.7, top-p 0.8) on both tasks, so they differ from the thinking-on rows in decoding as well as in mode.

Task	Condition	Correct %	Cut %	Close thinking	At once
BIRD, n=150	Base, thinking on	5.3	75.3	37	0
	Base, thinking off	6.7	0.7	–	–
	Authors' math edit	10.7	8.0	139	0
	Fable run 1 edit	2.0	4.7	144	140
	Fable run 2 edit	2.7	0.7	150	150
MATH-500, n=100	Base, thinking on	8	92	8	0
	Base, thinking off	42	33	–	–
	Authors' math edit	21	72	28	0
	Fable run 1 edit	9	91	9	0
	Fable run 2 edit	9	89	13	0

Table 3: The report's sampling protocol. BIRD at 8,192 tokens; MATH-500 at 16,384 tokens with thinking on and 8,192 with thinking off. Thinking-off rows use model-card sampling on both tasks. One seed.

6 What the trace says about the method

Understanding versus finding a lever. The session contains three mechanistic claims of the form "these components cause the loop; changing them will stop it". Each was specific, each was backed by a measurement the editor made, and each was turned into a precise edit and tested on held-out generation. The first two were false in the sense that matters: the predicted effect did not occur. That does not show the components are uninvolved; redundancy or compensation can hide a real role. The third was true: it is a correct mechanistic claim about entry into the thinking mode. What the method did not produce is a claim about what sustains the loop once entered; the edit prevents entry. The repair reading predicted that edits A and B would reduce looping; they did not. The search reading predicted that some small weight set would work regardless of the explanation; edit C did, and so did gradient descent on its footprint. The method's output here is a bypass, which the editor itself described accurately in its final answer: "skip the degenerate Thinking Process loop and answer directly".

Calibration. [TODO: table of every stated prediction with its outcome, from the turn logs: predicted direction, predicted size where given, observed.]

What the method did well. Finding four position-selective neurons among 14,336, choosing a direction in the residual stream that the output layer reads as one token, and setting four scalars so that the token wins by 2 nats, took the editor 17 forward passes and about 160 code runs, and the random-location control shows the result was not luck of the layer. The editor chose its own confirmation protocol, seven independent paired sets of 64, after the harness required nothing beyond one.

What it did not do. It did not repair the loop. An edit that ends the loop and keeps the reasoning exists: under sampling, the authors' math edit is one (Section 5). The editor did not find it, and two properties of our harness may have contributed. Every pass used greedy decoding, under which that repair is mostly invisible. And the success criterion was execution accuracy, which a bypass satisfies. We therefore cannot attribute the bypass to the editor alone. The ablation map will test whether any single head or 64-column MLP block reduces the loop rate under greedy decoding when switched off. [TODO: ablation map] It also did not find the authors' loop direction, and the authors' edit did not find this switch; the two edits are in different layers and act on different decisions.

Cost. At list prices the session used 17.3M cached input tokens and 493k output tokens, about $800; 9.8 hours of wall clock, of which about 6 were the laptop running forward passes. The equivalent template flag costs nothing.

7 Related work

Weight-space programming [1] is the direct subject. Model editing with ROME and MEMIT [11] locates and rewrites factual associations with a closed-form update; activation steering [12] adds a direction to the residual stream at inference time rather than in the weights. Edit C is a weight-space form of steering at a single position, gated by neurons that detect that position. The induction-head hypothesis the editor tried first follows [9]; direct logit attribution follows the logit lens [10]. The evaluation of thinking-mode loops in small reasoning models, and the finding

that "no-think" modes often match or beat them on small models, is documented in [2]. Public replications of the report [3, 4, 5, 6] show that the base model with thinking off recovers most of the math gain, and one blind attempt with a Claude editor on math found no working edit [6].

8 Limitations

One editor, two runs, one domain, one model size. The sampled results use one seed. Our MATH-500 runs use 100 problems and a 16,384-token limit, half the report's, so the math edit's 21% is not a replication of the published 48%. The thinking-off math run used the model card's sampling settings and an 8,192-token limit, and 33% of its answers were cut. The held-out set is 150 questions, so accuracy differences of a few points are within noise; the loop and cut rates are not. The random-location rows use 50 questions each, and they test arbitrary neurons only; activation-matched neurons and random directions at the chosen neurons are not tested. The grader compares result sets, ignoring order and multiplicity, and takes SQL from a cut thinking block when one is present: 3 of the base model's 8 correct sampled answers never closed thinking. Practice is Spider and the test is BIRD, a shift that explains part of the gap between the 28% practice accuracy and the 6% test accuracy. The harness is a re-implementation from the report's released prompts and logs, not the authors' code. Both editor sessions and all controls ran under greedy decoding, chosen so that paired comparisons are exact. Section 5 shows that this choice hides the math edit's effect on SQL, so we cannot say whether an editor scored under sampling, or scored with a criterion that rejects answers without reasoning, would have found a repair. The editor is a Claude model and this paper was drafted with help from the same model family; the harness, controls and numbers are released so that the reader does not need to trust either. The editor may know the report, which was public before the session; the trace shows it rediscovered layer 15 head 7 from attention data and that this did not help, but we cannot exclude that prior knowledge shaped its search.

9 Conclusion

When one model debugs another by editing its weights, a loop that disappears can mean two things, and the difference is measurable from the outputs alone. On text-to-SQL, two independent frontier-editor sessions and two gradient baselines all produced a bypass, equal in effect to disabling thinking in the chat template; two attempts at a mechanistic repair failed. The published math edit is a repair by the same measurement, and the two outcomes differ in a second way: the repair, written for math, also removes the SQL loop under the report's sampling settings, and the bypass, written for SQL, does nothing on math. The task therefore does not force a bypass. What remains open is why the editor produced one: the editor itself, greedy scoring, or a success criterion that a bypass satisfies. Next: a session in which an answer without reasoning does not count and passes are scored under sampling; two GPT-6 Astra sessions on SQL, which hold the task fixed and change the editor; and an ablation map over every attention head and every 64-column MLP block.

Reproduction

Edit files, the 328 turn logs, the editor's notes, the harness and the evaluation scripts are at `https://github.com/Exorust/wsp-sql`. The edit is four `.npy` files totalling 16 KB; applying it to the public checkpoint and running `eval_sql.py` on BIRD Mini-Dev reproduces the edit C row of Table 1 in about 20 minutes on an Apple M-series laptop.

References

[1] R. Takehi, J. Teoh, Y. Zhang, O. Khattab. Sparks of Weight-Space Programming: LLMs can understand and write neural network weights. MIT CSAIL, 2 November 2026. [TODO: URL]

[2] Qwen Team. Qwen3.5 technical report, 2026. [TODO: exact reference]

[3] Hedwig AI. Does a weight edit teach Qwen3.5-0.8B math, or stop it looping? November 2026. https://www.hedwigai.com/blog/weight-edit-math-or-loops

[4] JonathanColetti (Hugging Face user). Qwen3.5-0.8B-Astra-WeightEdit, model card with independent benchmark. Hugging Face, November 2026.

[5] shapor (GitHub user). weight-space-experiments. GitHub, November 2026. https://github.com/shapor/weight-space-experiments

[6] kyle-chalmers (GitHub user). weight-space-programming. GitHub, November 2026. https://github.com/kyle-chalmers/weight-space-programming

[7] J. Li et al. Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. NeurIPS 2023.

[8] T. Yu et al. Spider: a large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. EMNLP 2018.

[9] C. Olsson et al. In-context learning and induction heads. Transformer Circuits Thread, 2022.

[10] nostalgebraist. Interpreting GPT: the logit lens. 2020.

[11] K. Meng, D. Bau, A. Andonian, Y. Belinkov. Locating and editing factual associations in GPT. NeurIPS 2022.

[12] A. Turner et al. Activation addition: steering language models without optimization. 2023.

A The edit

For layer $\ell \in \{17, 18, 19, 20\}$ with neuron $j_\ell \in \{537, 628, 2424, 1235\}$ and $\alpha_\ell \in \{0.20, 0.20, 0.25, 0.15\}$:

$$W_\ell^{\text{down}}[:, j_\ell] \leftarrow W_\ell^{\text{down}}[:, j_\ell] + \alpha_\ell \, u, \qquad u = \frac{\gamma \odot E[\texttt{</think>}]}{\|\gamma \odot E[\texttt{</think>}]\|},$$

with γ the final RMSNorm gain and E the tied embedding matrix. Entries whose bfloat16 value did not change after the cast are not counted; 4,083 of the 4,096 entries changed.

B Session statistics

Started 2026-11-06 18:13, ended 2026-11-07 04:03 local time. 328 turns, 299 of 300 code runs, 25 of 25 passes, 9 handoffs. Ten forward requests were rejected by the harness without counting against the budget (eight that tried to reference a file instead of inline JSON, two with invalid JSON). The editor recorded pass 3, a 6-text teacher-forced read, as wasted. The kernel restarts at every handoff by design, so the editor recovered its state from its notes file nine times; its edit files persist across restarts.